

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

CS 498: Batch Serving Techniques

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy, and many colleagues

Today

LLM Serving

- Basics (Prefilling and Decoding)
- Continuous Batching
- Chunked-Prefill

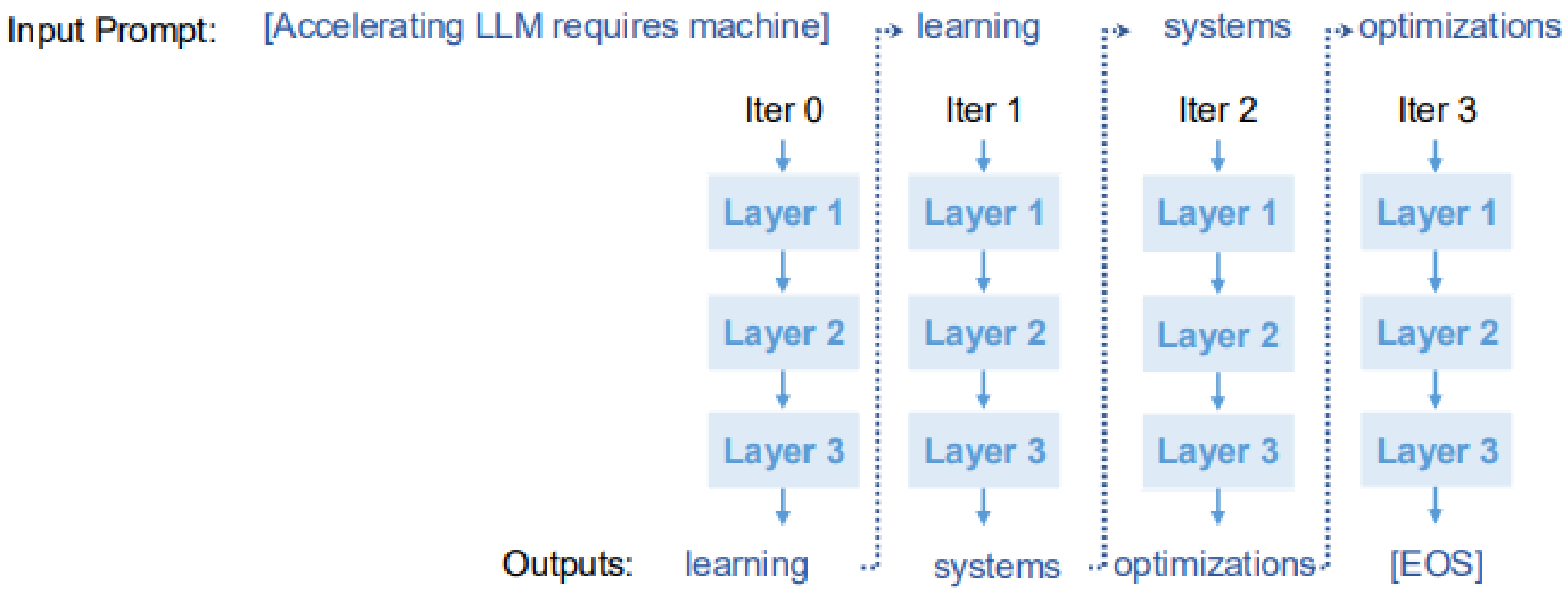
Panel Discussion (Speculative Decoding)

LLMs are Slow and Expensive to Serve



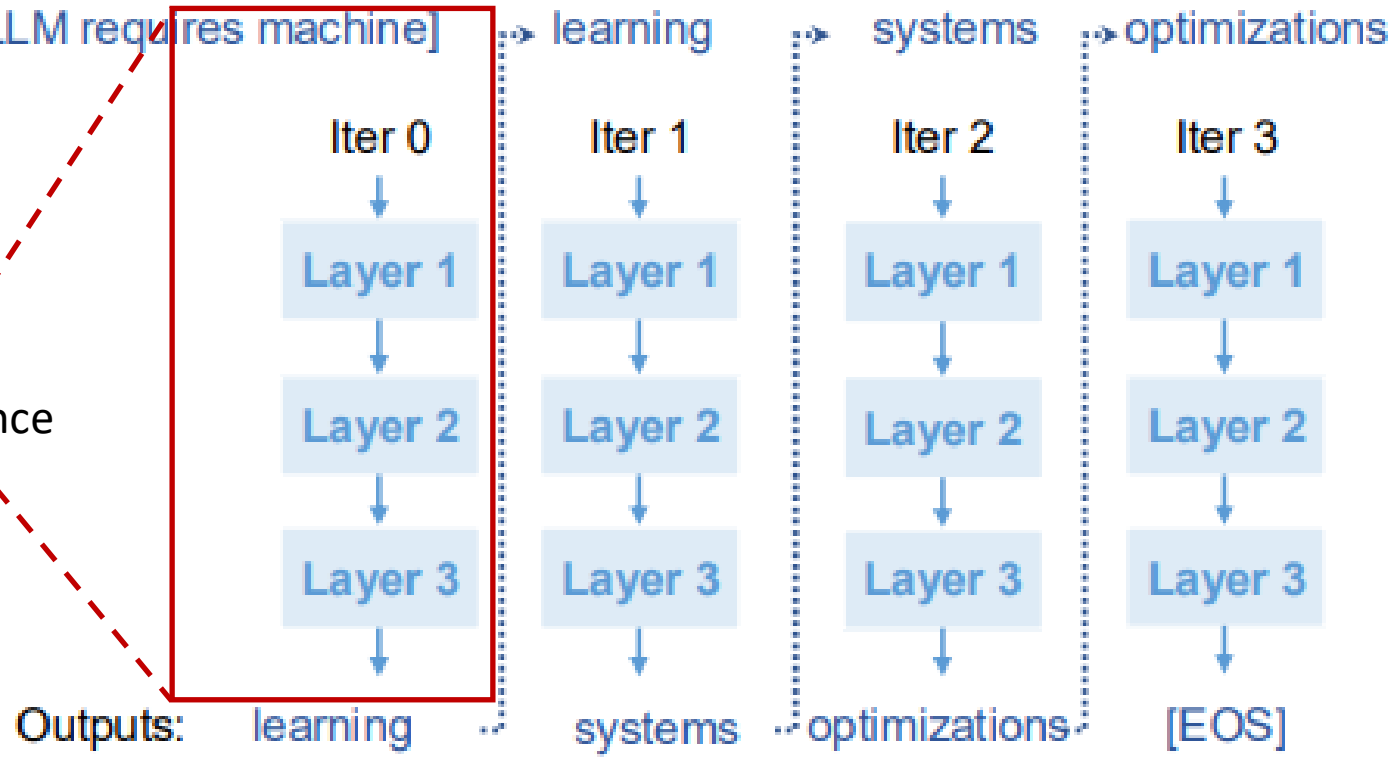
- At least ten A100-40GB GPUs to serve 175B GPT-3 in half-precision
- Generating 1024 tokens takes ~10 seconds
- Cannot process requests in parallel
  - Per-request key-value cache takes 3GB GPU memory

Generative LLM Inference: Autoregressive Decoding



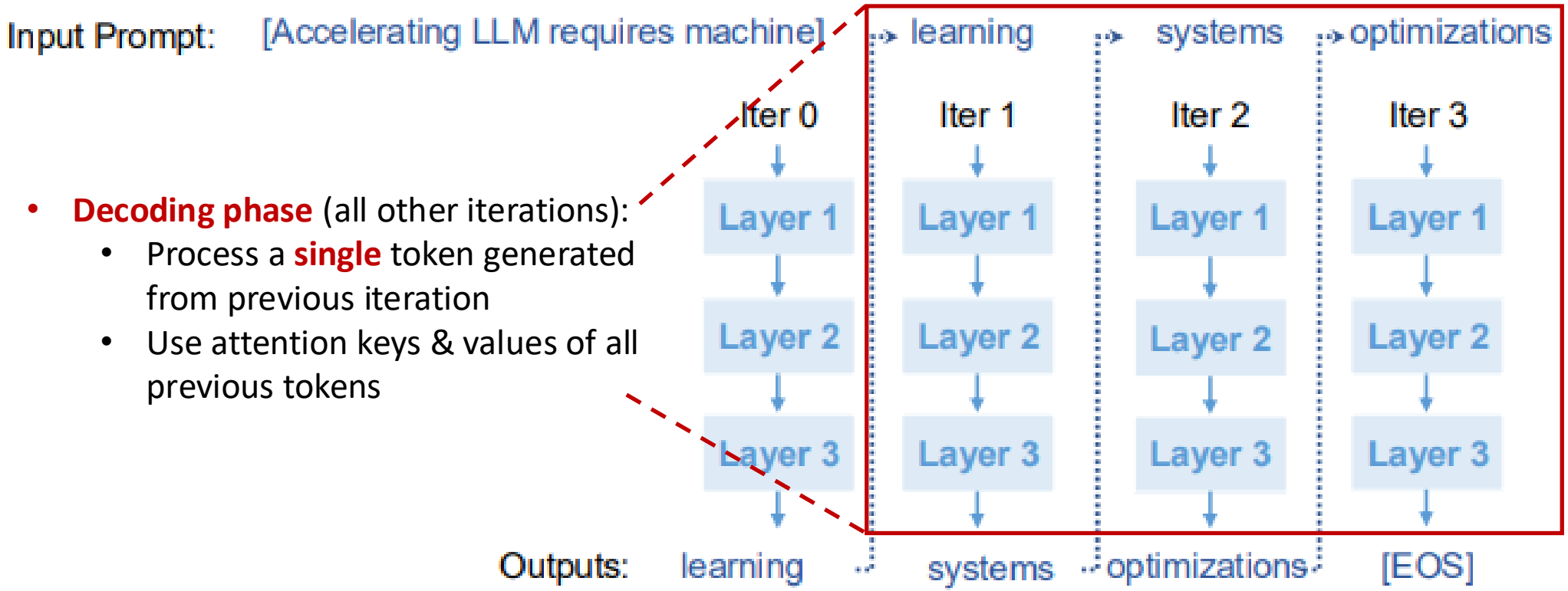
Generative LLM Inference: Autoregressive Decoding

- **Pre-filling phase** (0-th iteration):
  - Process **all** input tokens at once

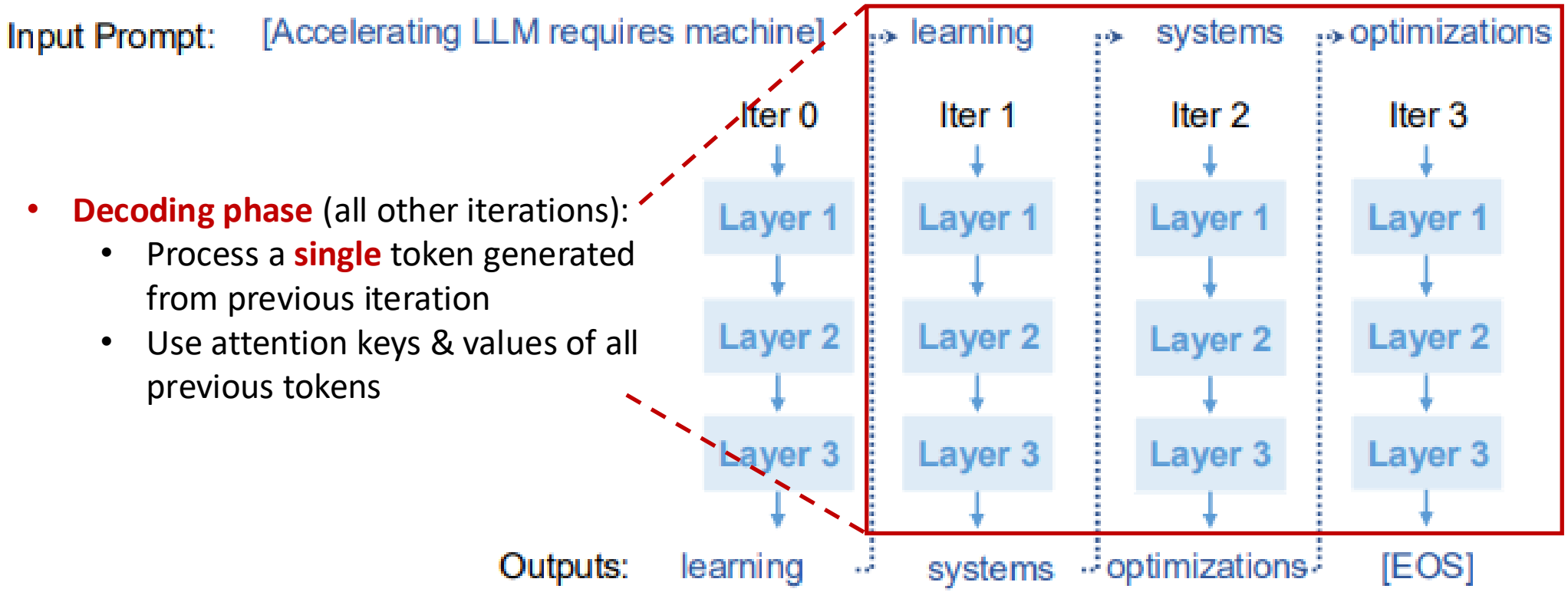


Generative LLM Inference: Autoregressive Decoding

- **Decoding phase** (all other iterations):
  - Process a **single** token generated from previous iteration
  - Use attention keys & values of all previous tokens



Generative LLM Inference: Autoregressive Decoding



- **Decoding phase** (all other iterations):
  - Process a **single** token generated from previous iteration
  - Use attention keys & values of all previous tokens

Repeat until the sequence

- Reaches the pre-defined maximum length (e.g., 2048 tokens)
- Generates the stop tokens (e.g., “<|end of sequence|>”)

Generative LLM Inference: Important Metrics

- **Time to First Token (TTFT)**: Measures how quickly users begin to see the model output token after submitting a query.
  - Critical for real-time interactions
  - Driven by prompt processing time and the generation of the first token
- **Time per Output Token (TPOT):** Time taken to generate each output token.
  - Impacts user perception of speed (e.g., 100ms/token = 10 tokens/second)
- **E2E Latency = TTFT + TPOT * (the number of generated tokens - 1)**
  - Total time to generate the complete response
- **Throughput:** Number of tokens generated per second across all requests by the inference server

LLM Inference Performance Engineering: Best Practices | Databricks

Optimizing LLM Inference: Goals and Tradeoffs

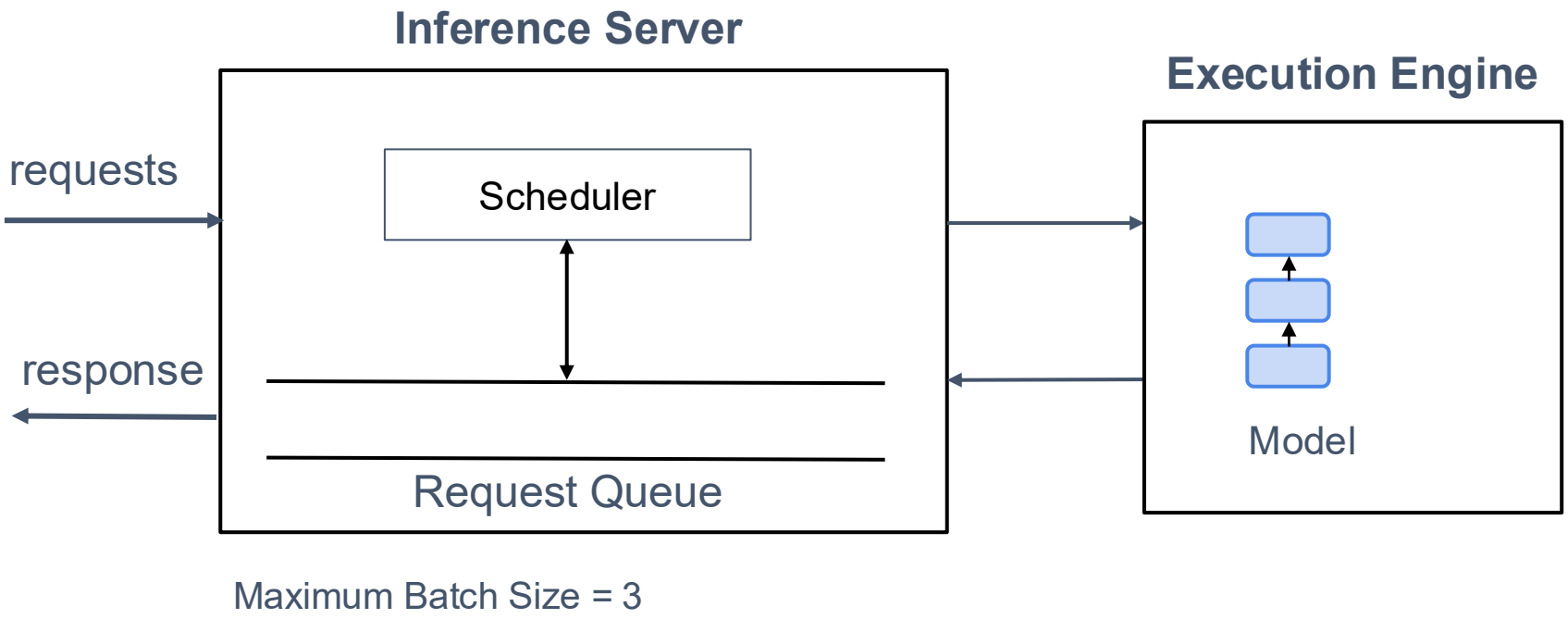
- **Goal:** Minimize TTFT, maximize throughput, and reduce TPOT
- **Throughput vs. TPOP Tradeoff**: Processing multiple queries concurrently increases throughput extends TPOT for each user.

Today

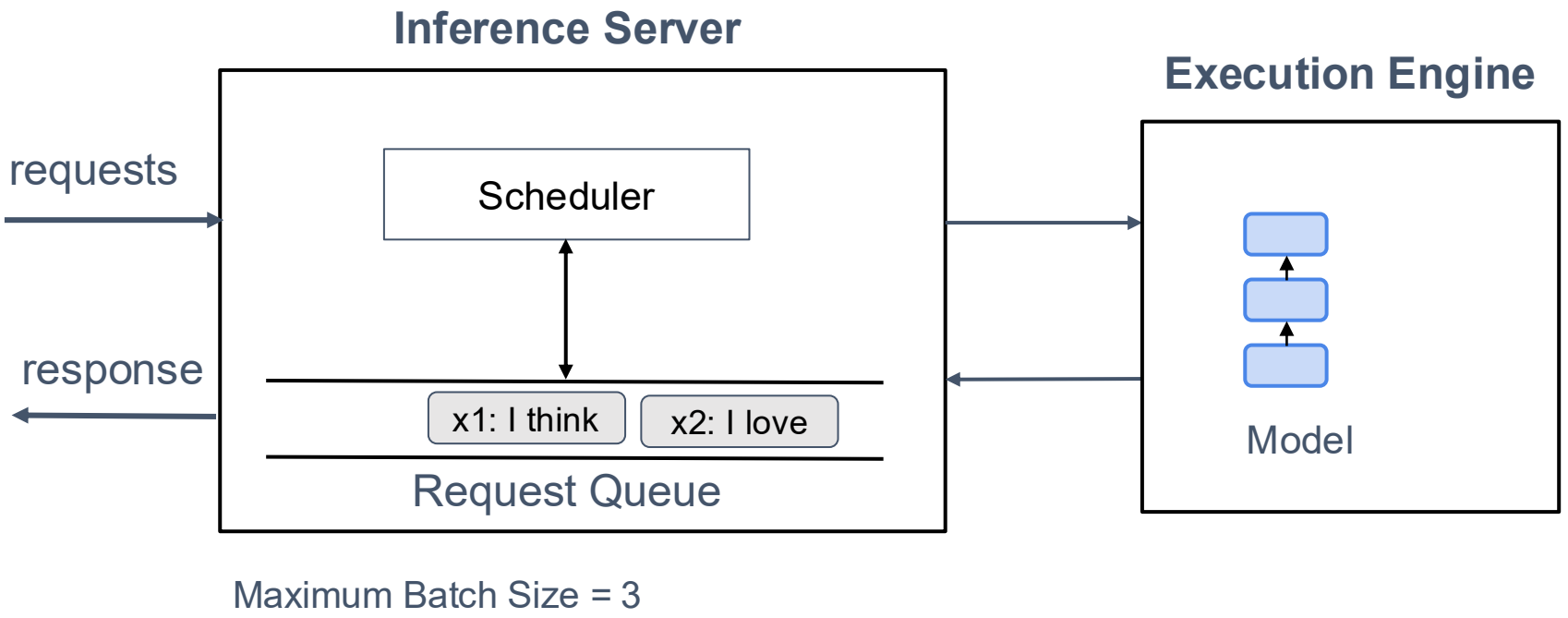
LLM Inference

- Basics (Prefilling and Decoding)
- Continuous Batching
- Chunked-Prefill

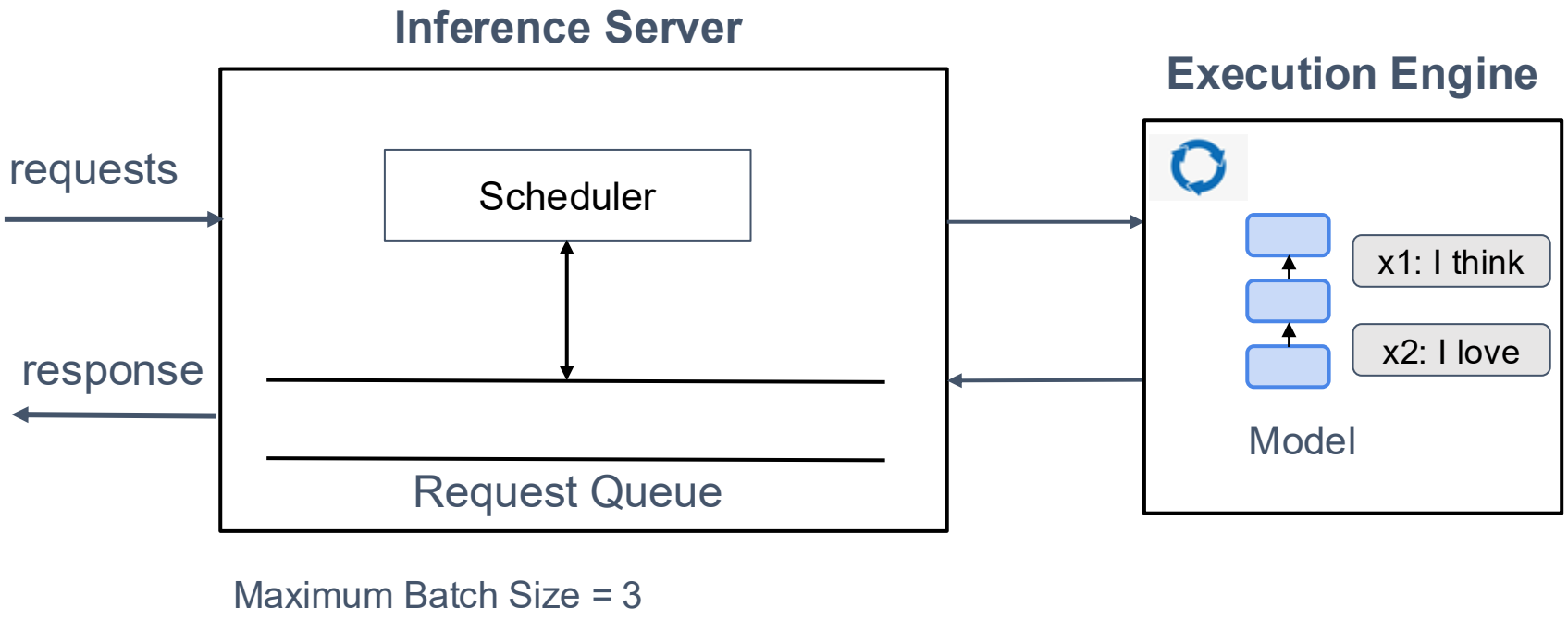
LLM Serving System



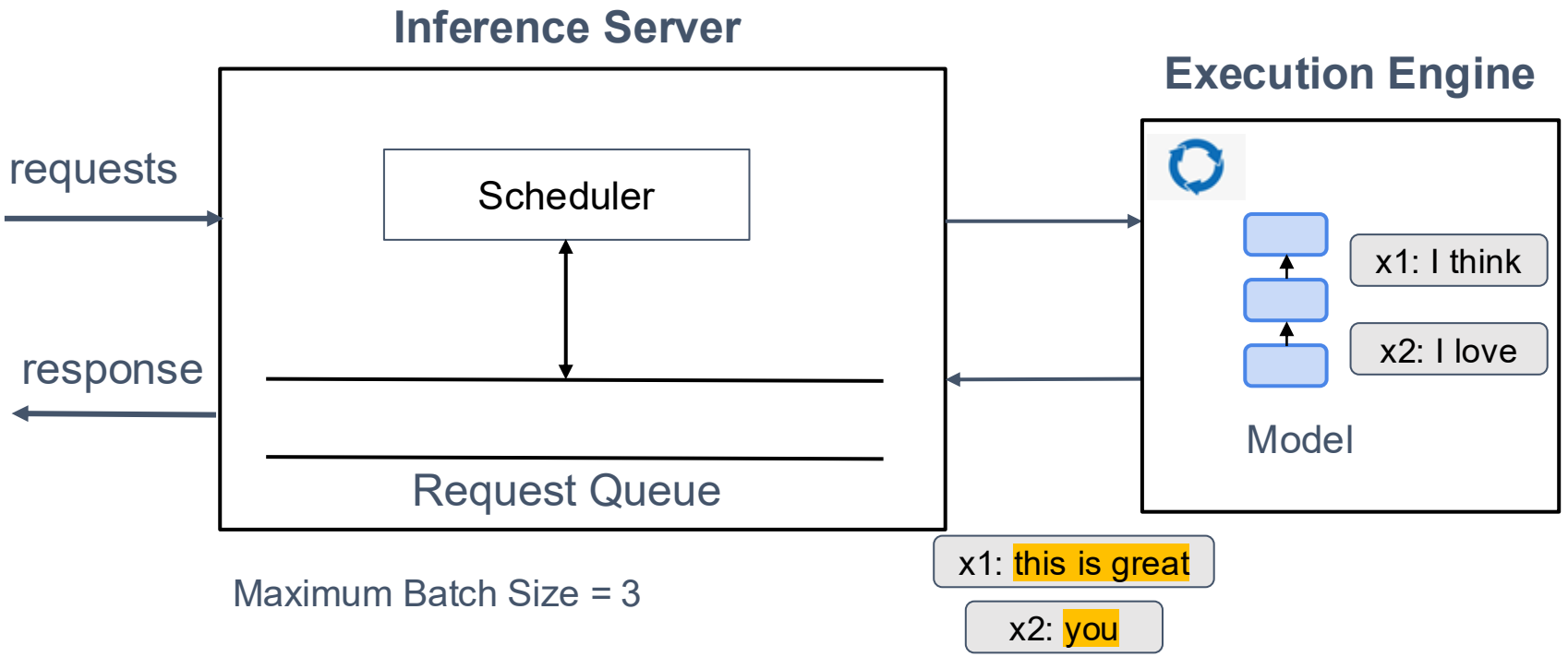
LLM Serving System



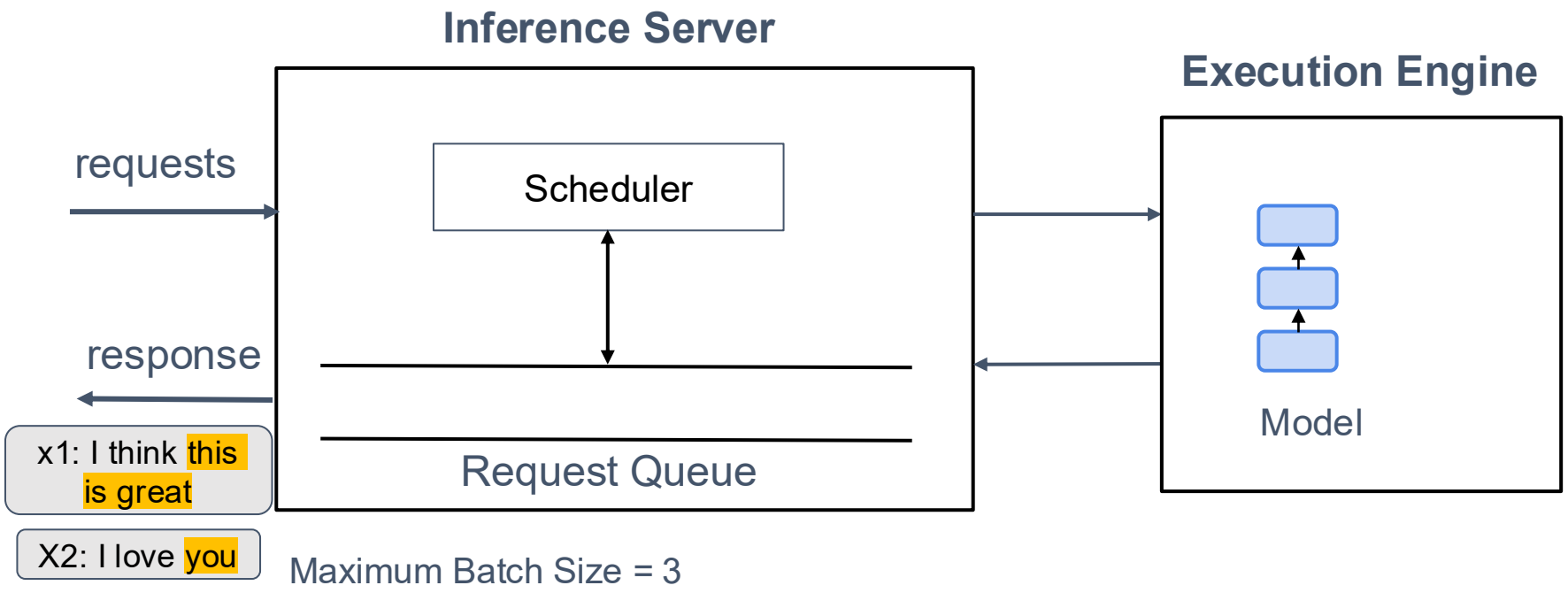
LLM Serving System



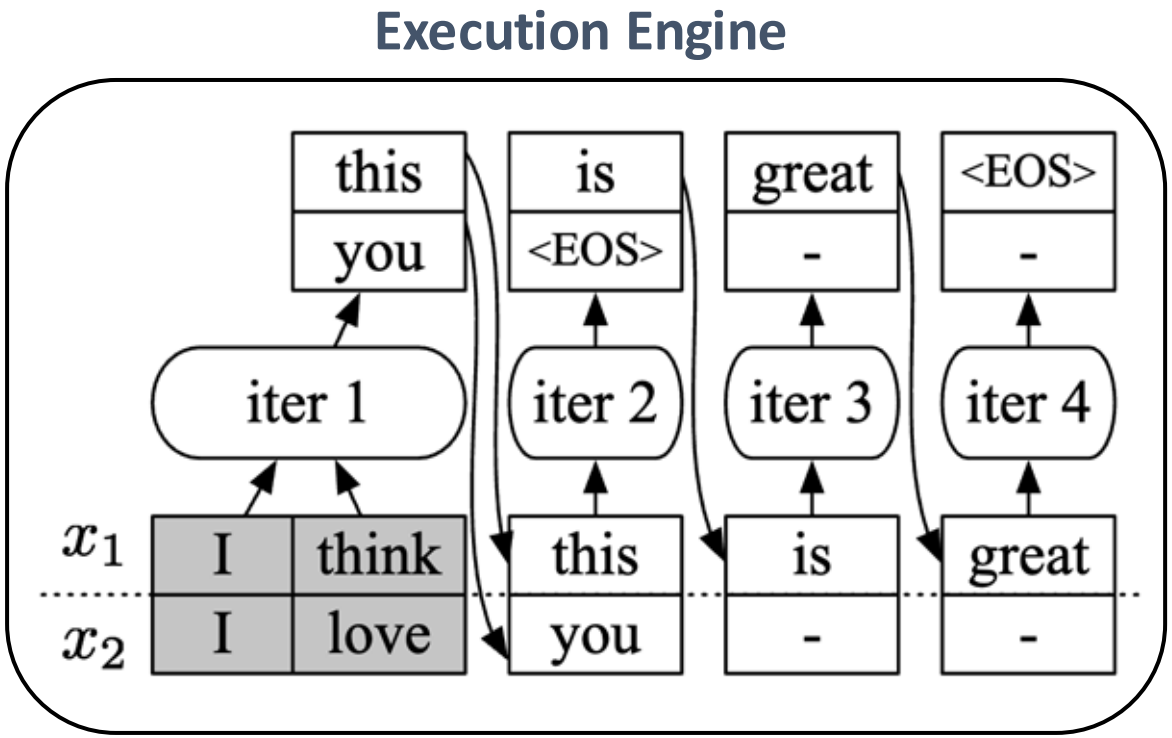
LLM Serving System



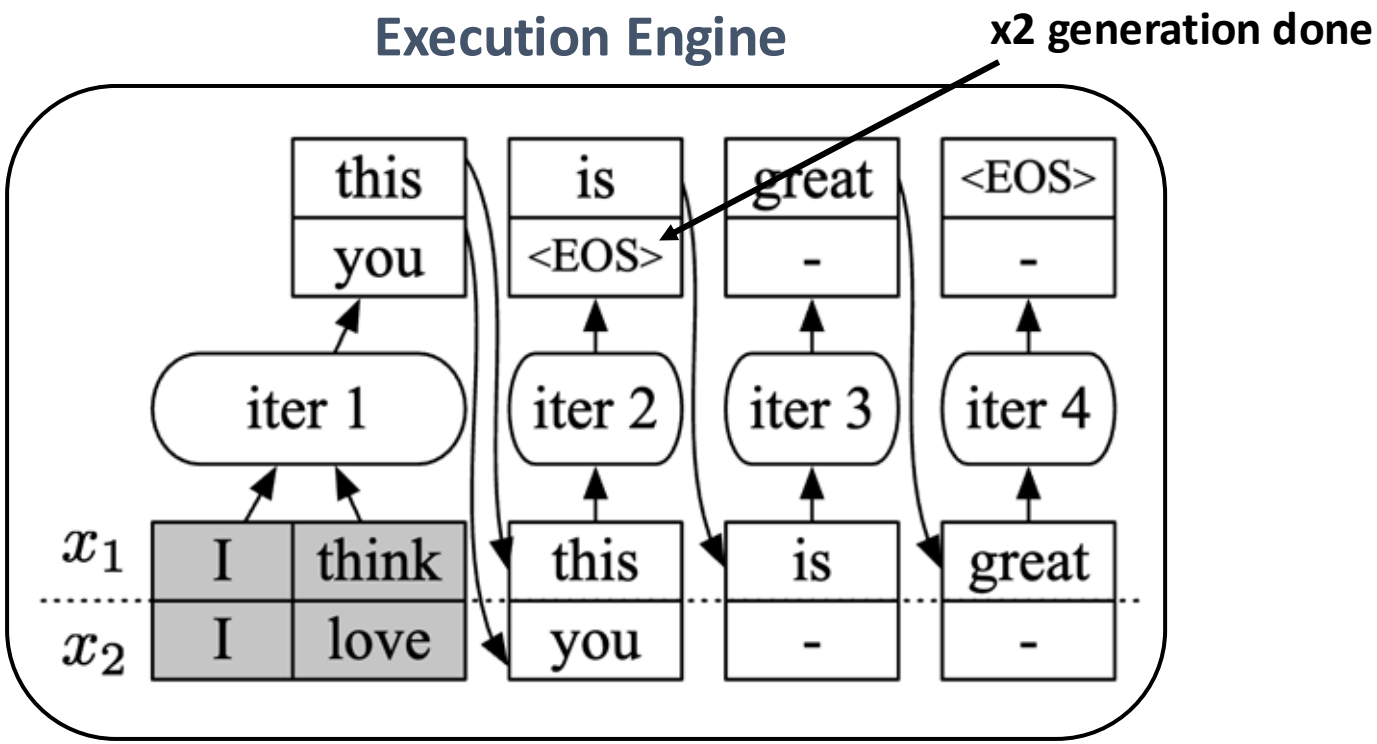
LLM Serving System



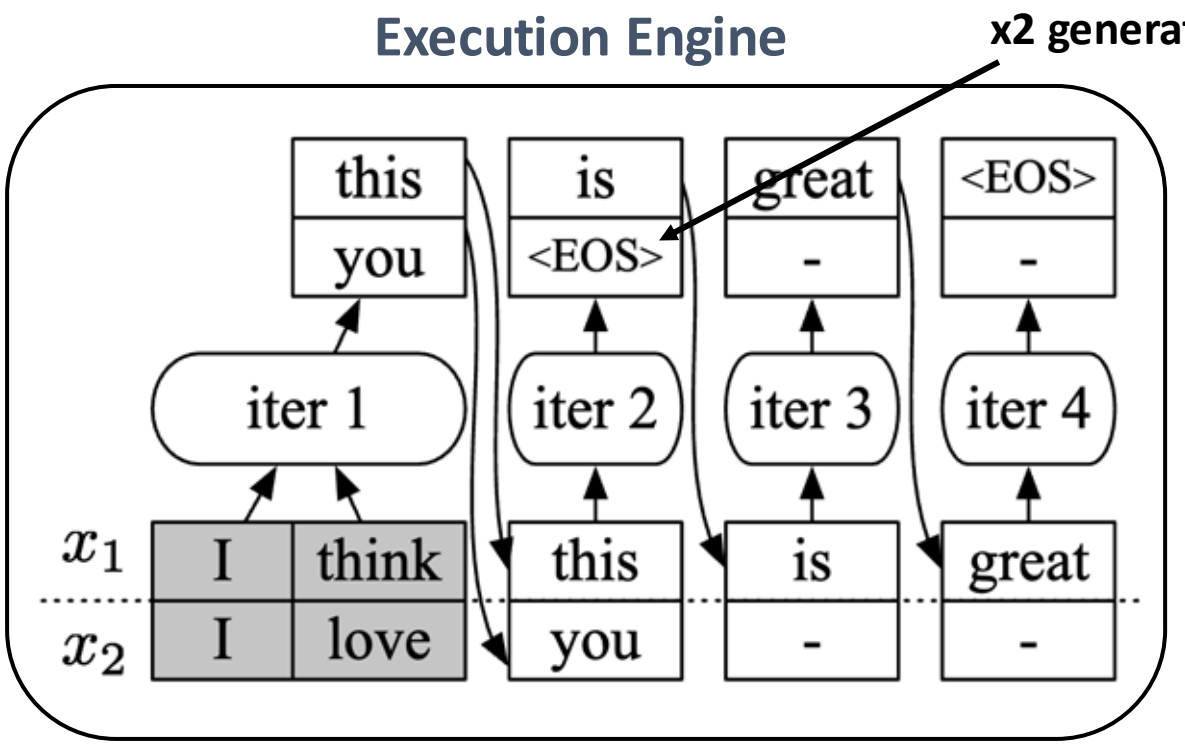
Problem 1: Request Level Scheduling



Problem 1: Request Level Scheduling

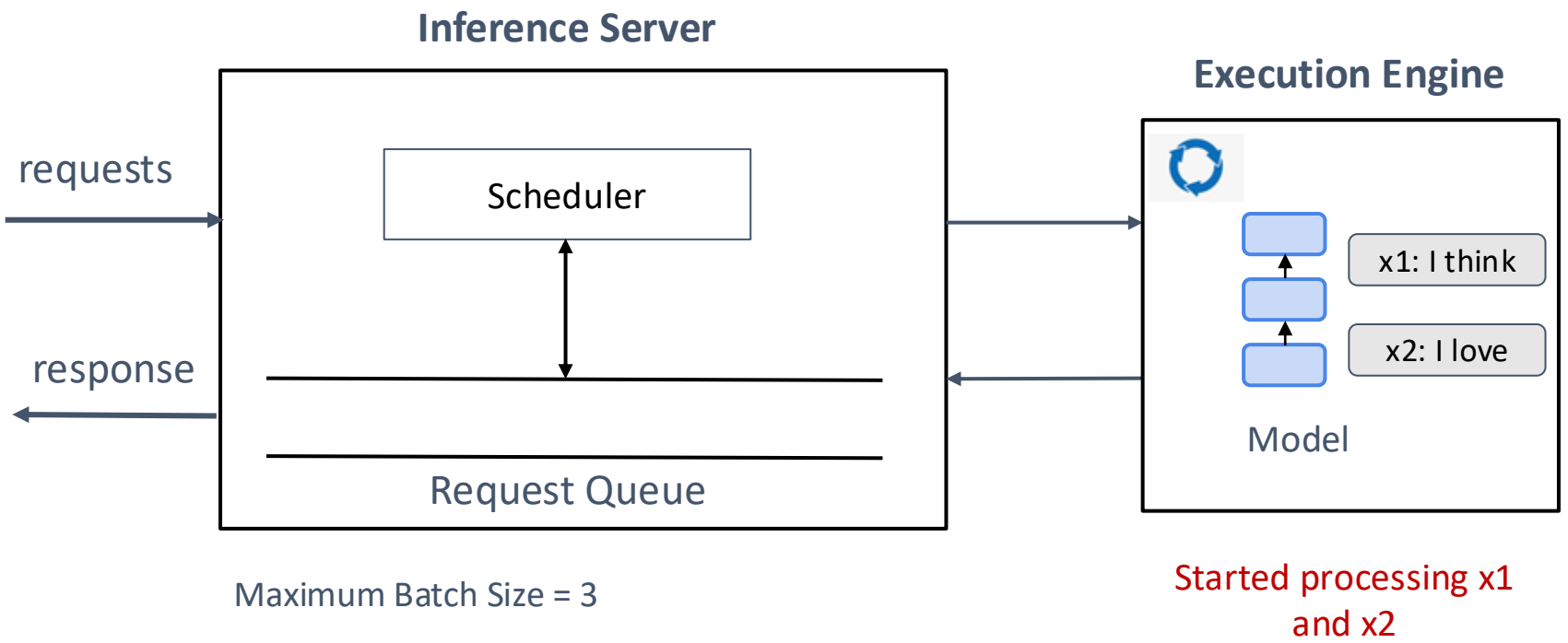


Problem 1: Request Level Scheduling

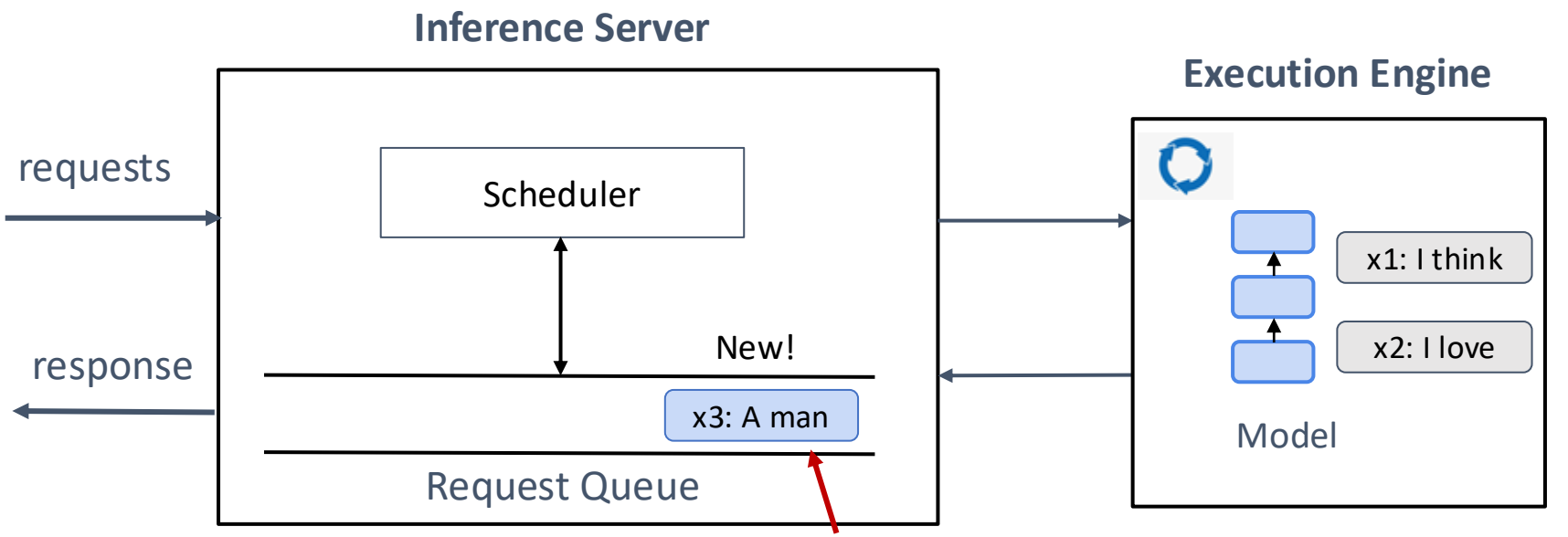


Early finished requests cannot return to the Inference Server → Latency increase

Problem 1: Request Level Scheduling



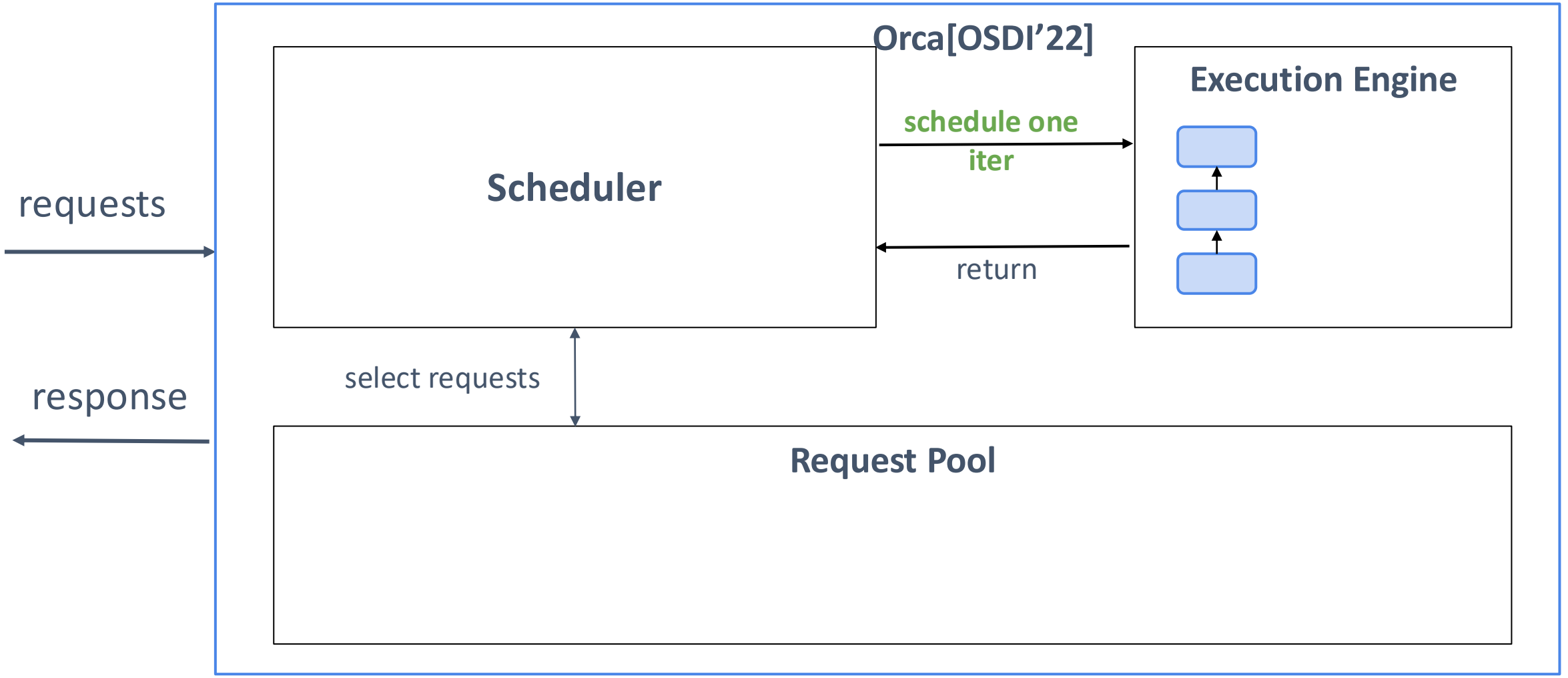
Problem 1: Request Level Scheduling



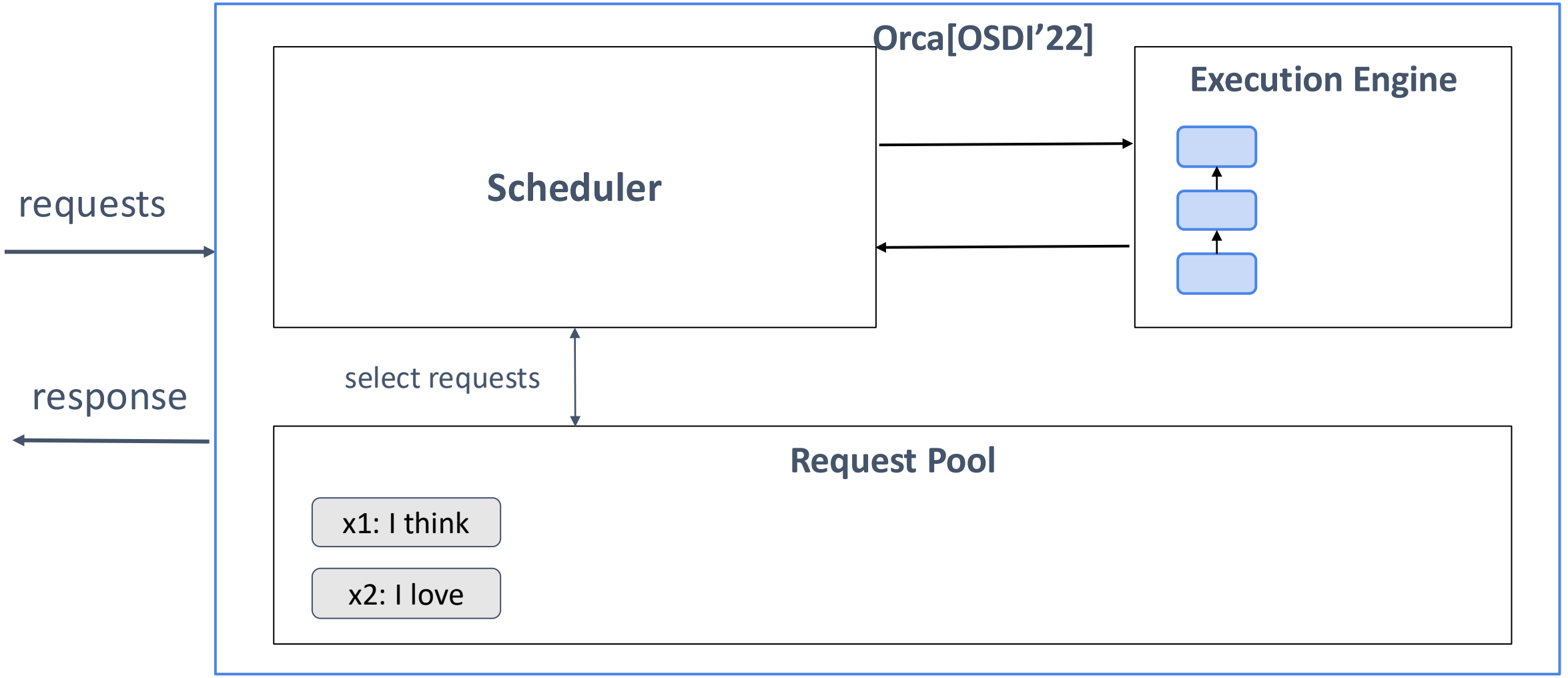
Late join requests need to wait until engine finishes execution

→ Latency Increase

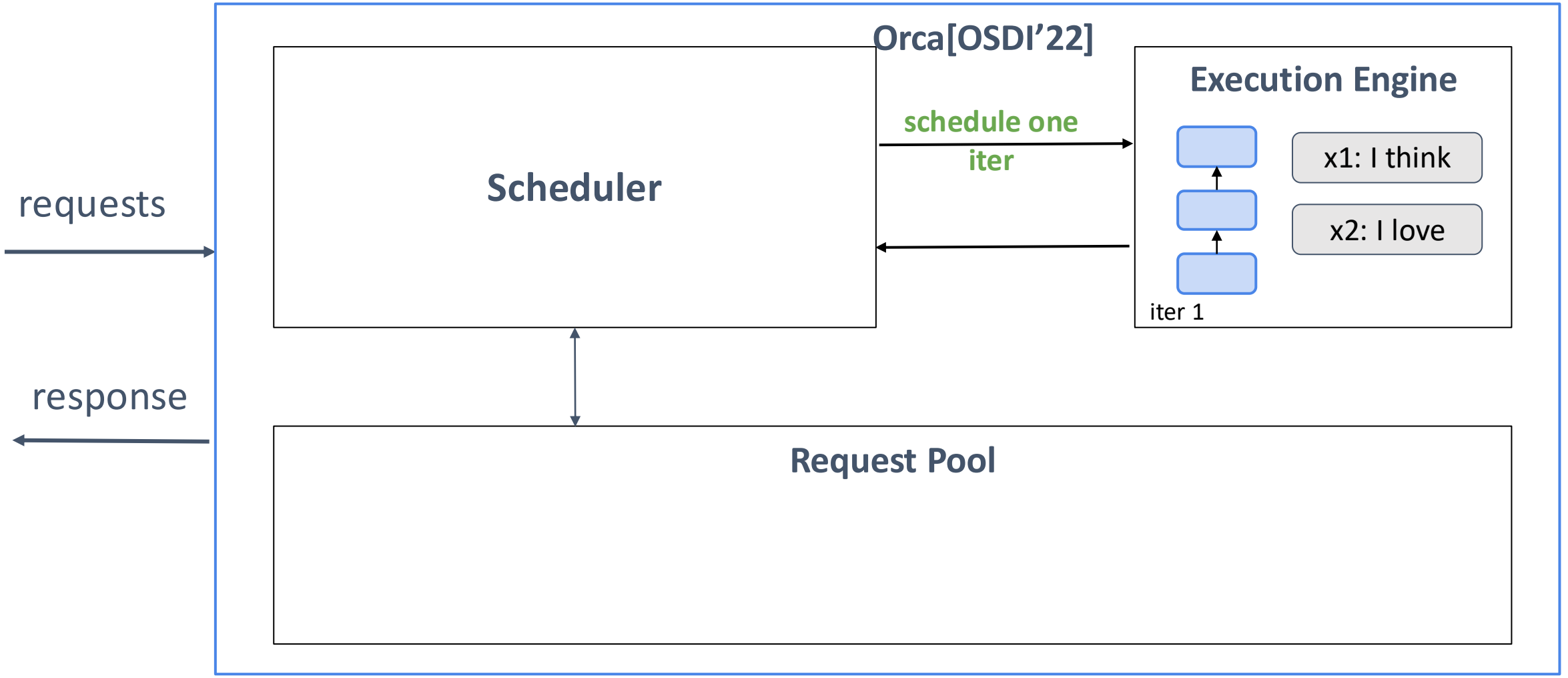
Solution 1: Iteration Level Scheduling



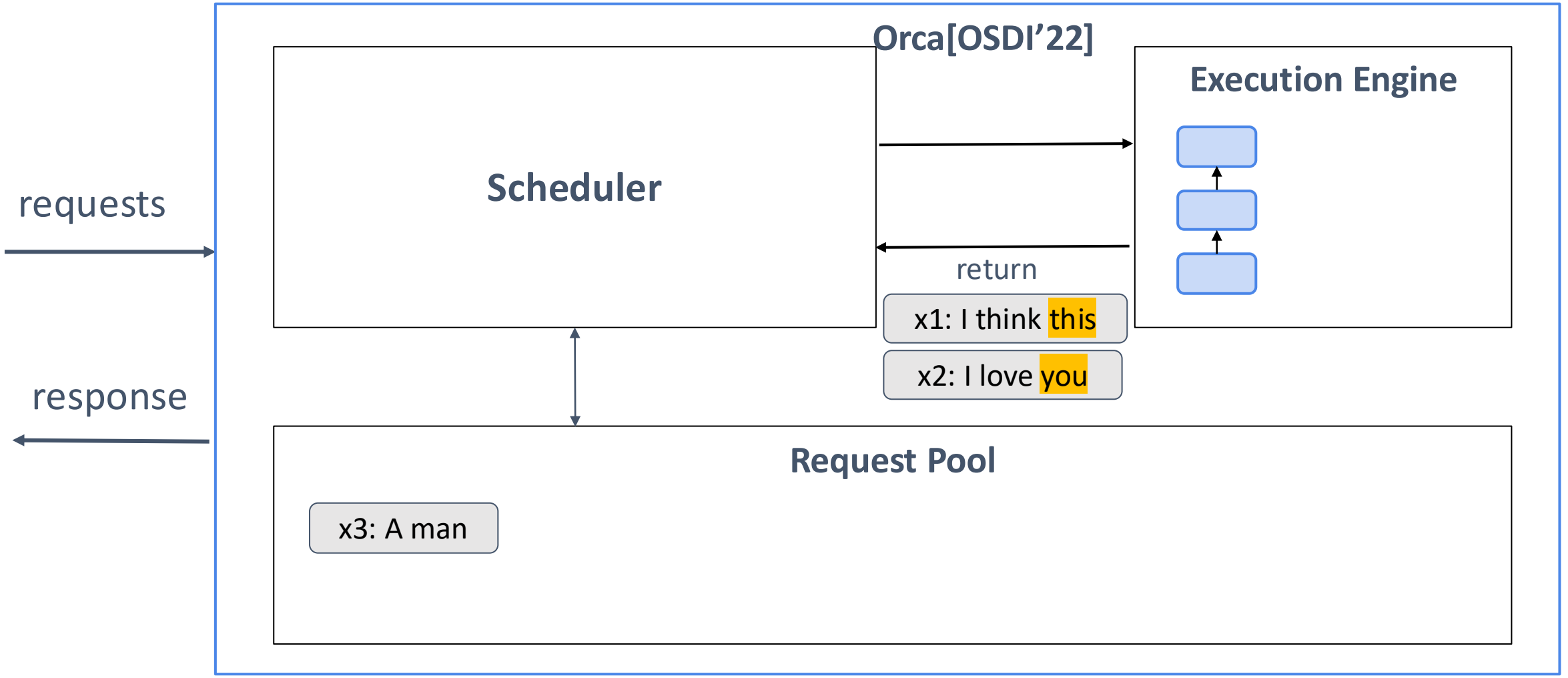
Solution 1: Iteration Level Scheduling



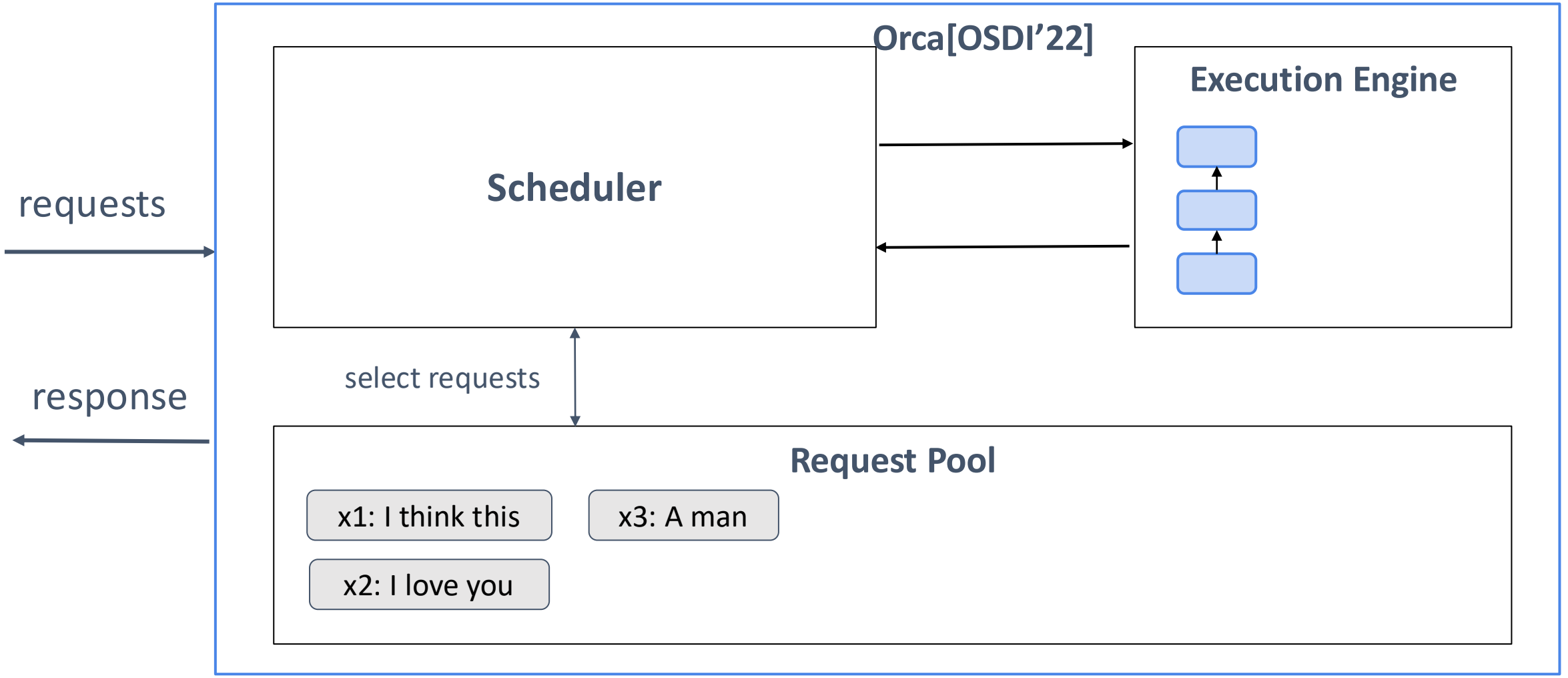
Solution 1: Iteration Level Scheduling



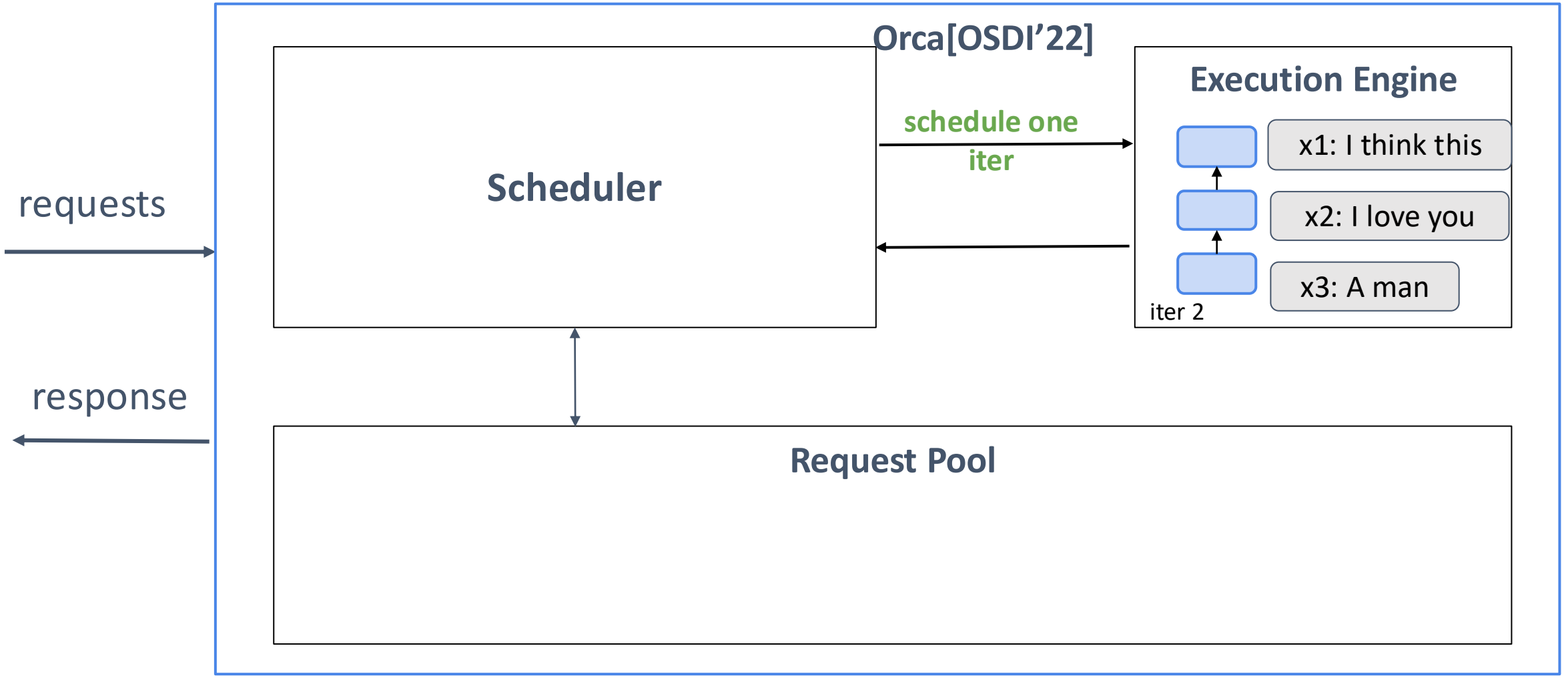
Solution 1: Iteration Level Scheduling



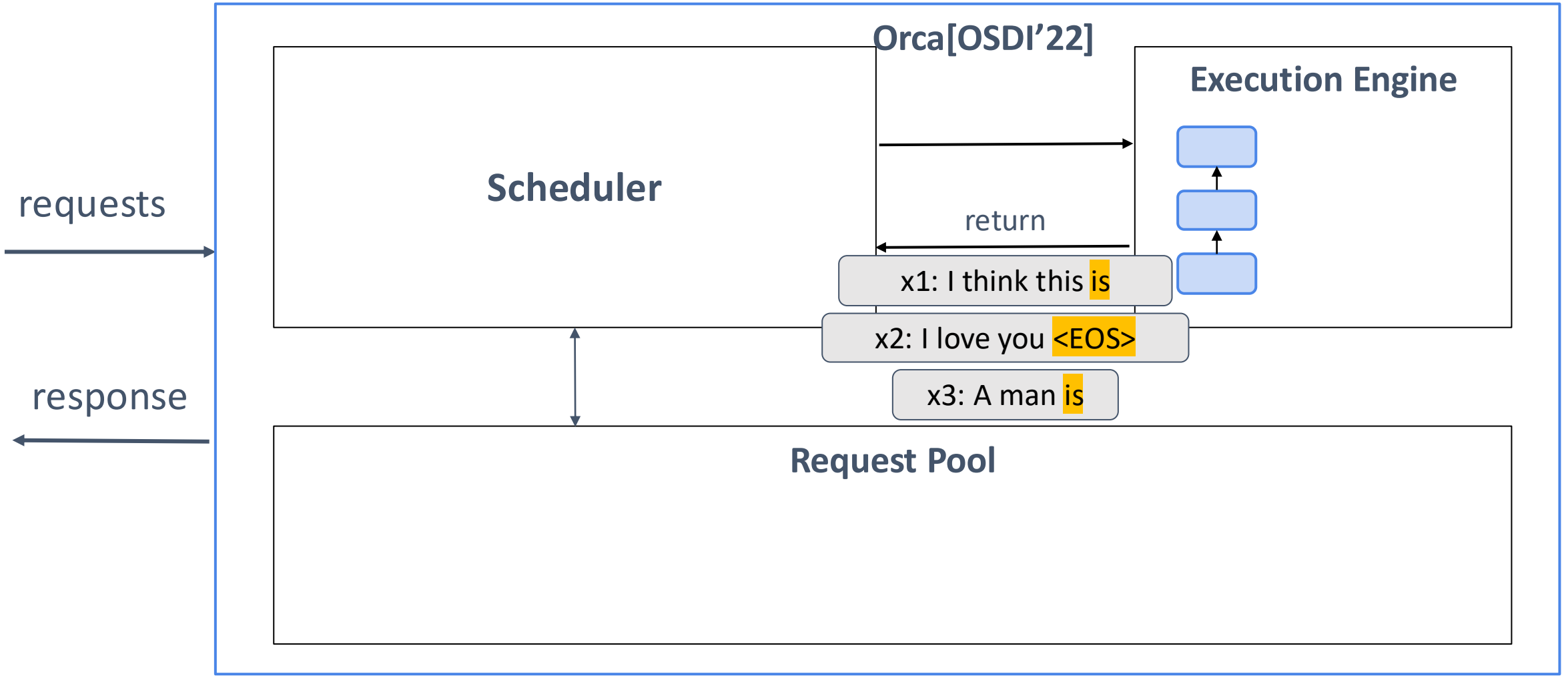
Solution 1: Iteration Level Scheduling



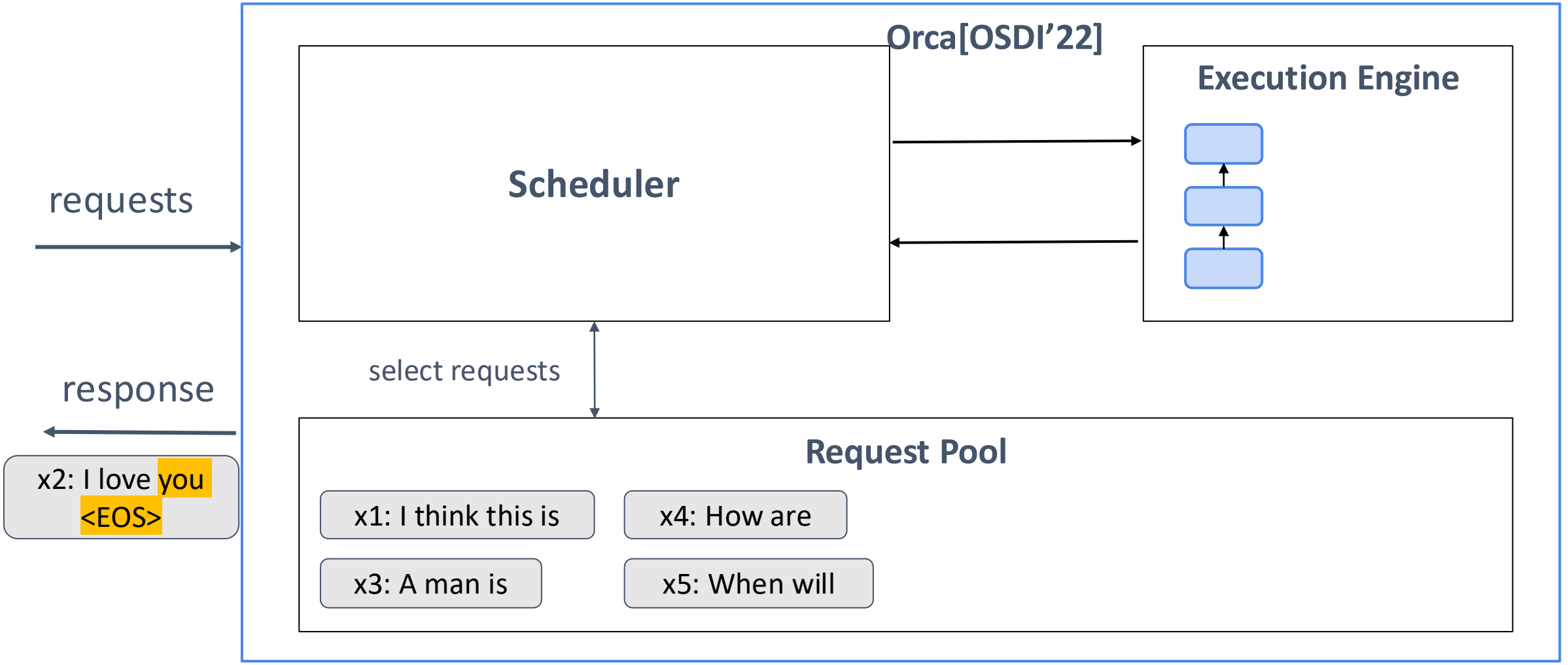
Solution 1: Iteration Level Scheduling



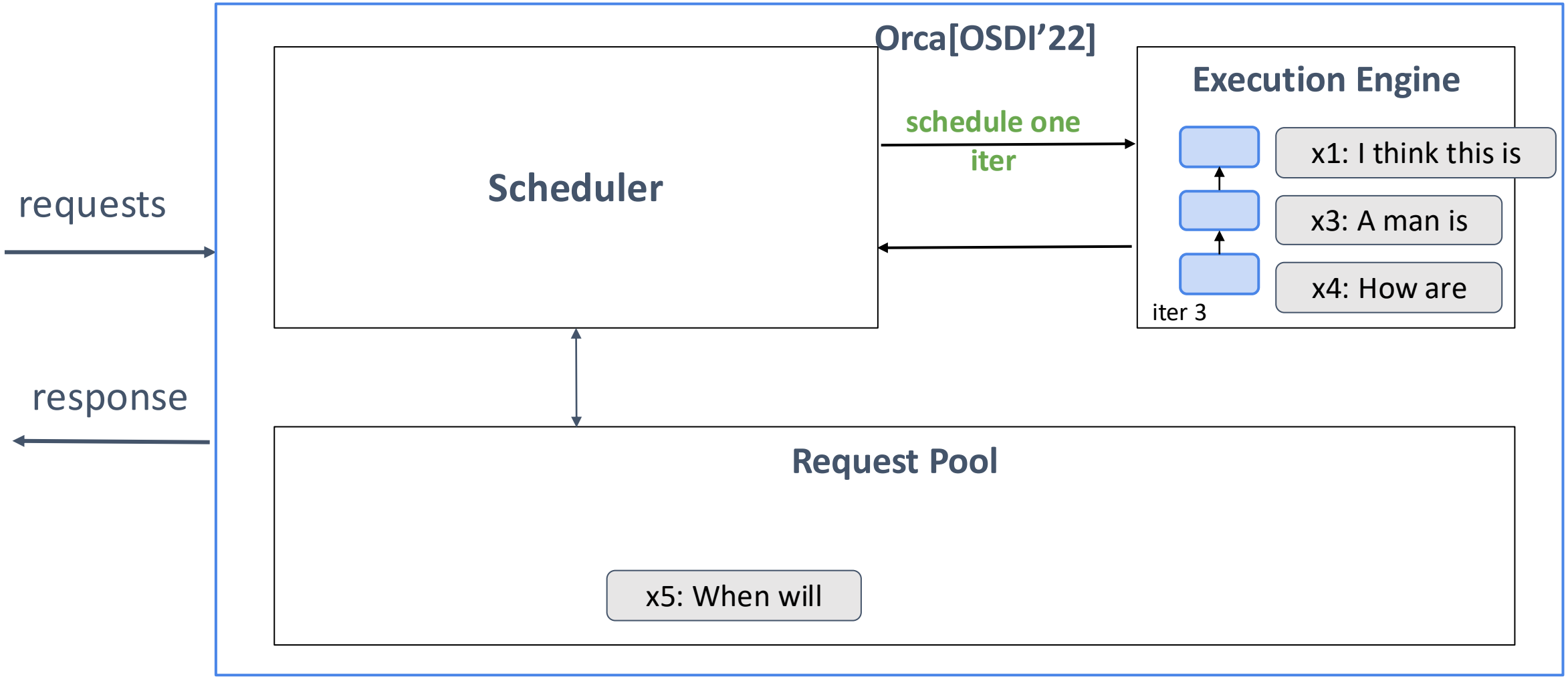
Solution 1: Iteration Level Scheduling



Solution 1: Iteration Level Scheduling



Solution 1: Iteration Level Scheduling



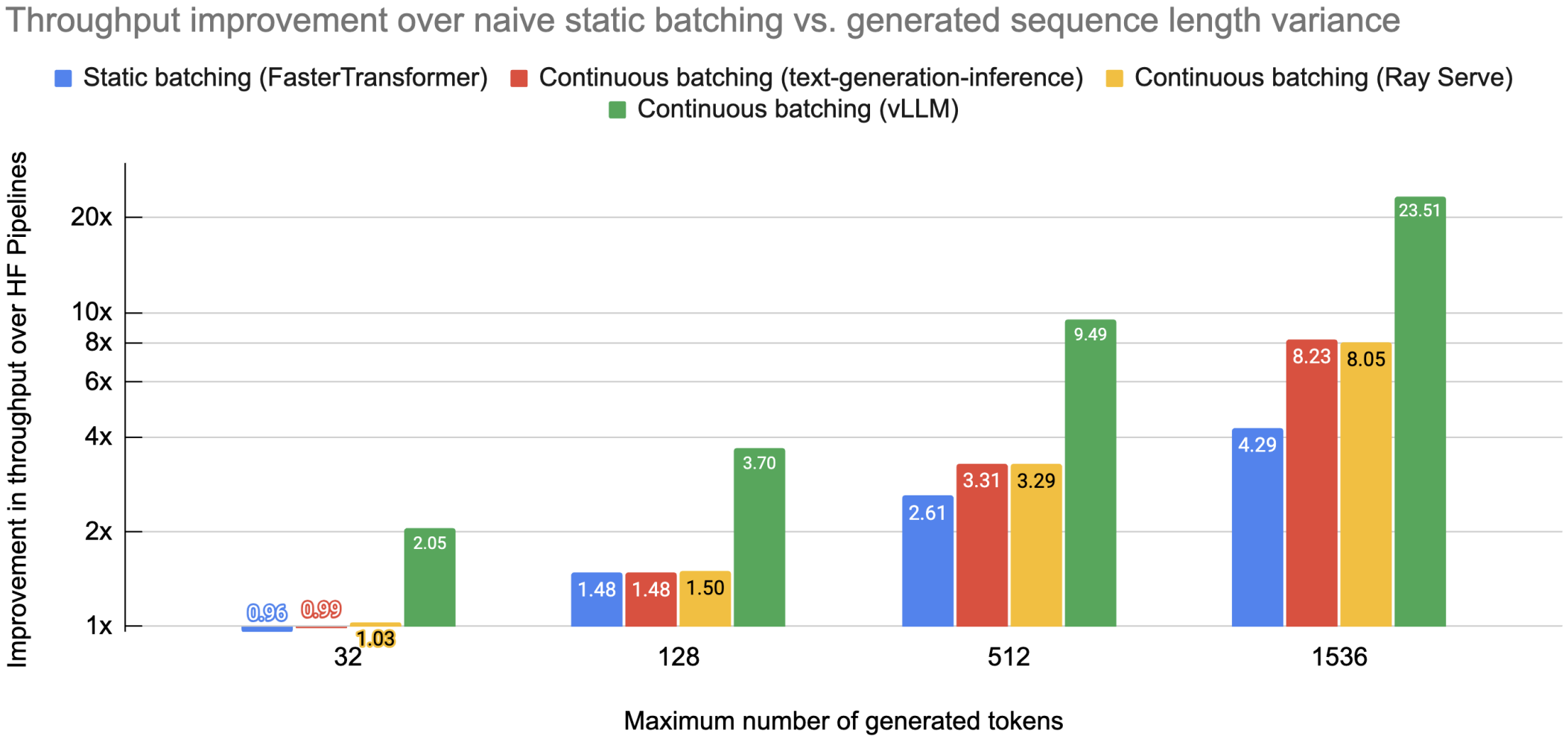
Solution 1: Iteration Level Scheduling



LLM Inference Scheduler

- Enforces iteration-level **first-come-first-served (FCFS)** property
- Maximum batch size → Throughput vs. Latency control knob
- Keep track of number of reserved slots to avoid deadlock
  - Slot := memory required for storing an Attention key and value for a single token

Throughput Improvement from Continuous Batching



https://www.anyscale.com/blog/continuous-batching-llm-inference

Summary: Continuous Batching

- Continuous batching handles early-finished and late-arrived requests more efficiently
- Fills GPU capacity after each token generation
- As variance in sequence length increases, continuous batching increases GPU utilization

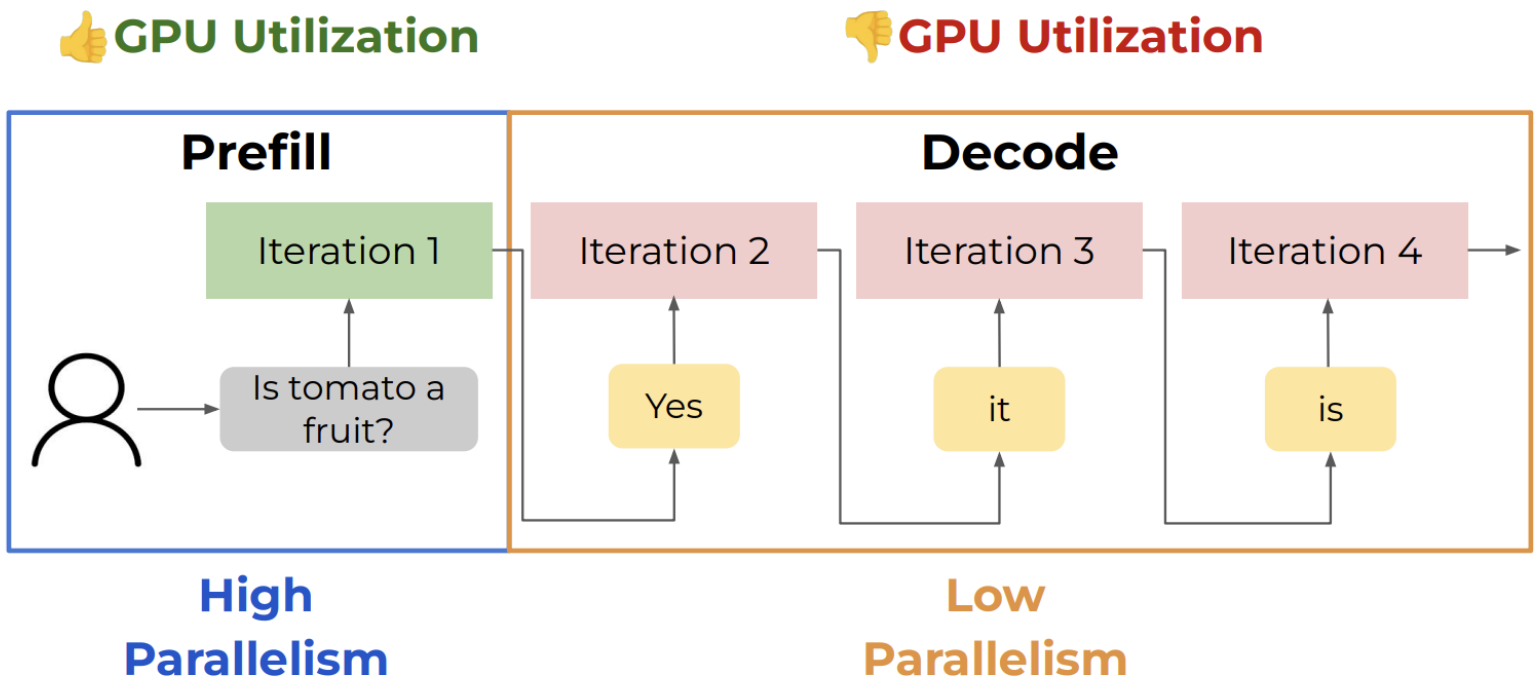
Today

LLM Inference

- Basics (Prefilling and Decoding)
- Continuous Batching
- Chunked-Prefill

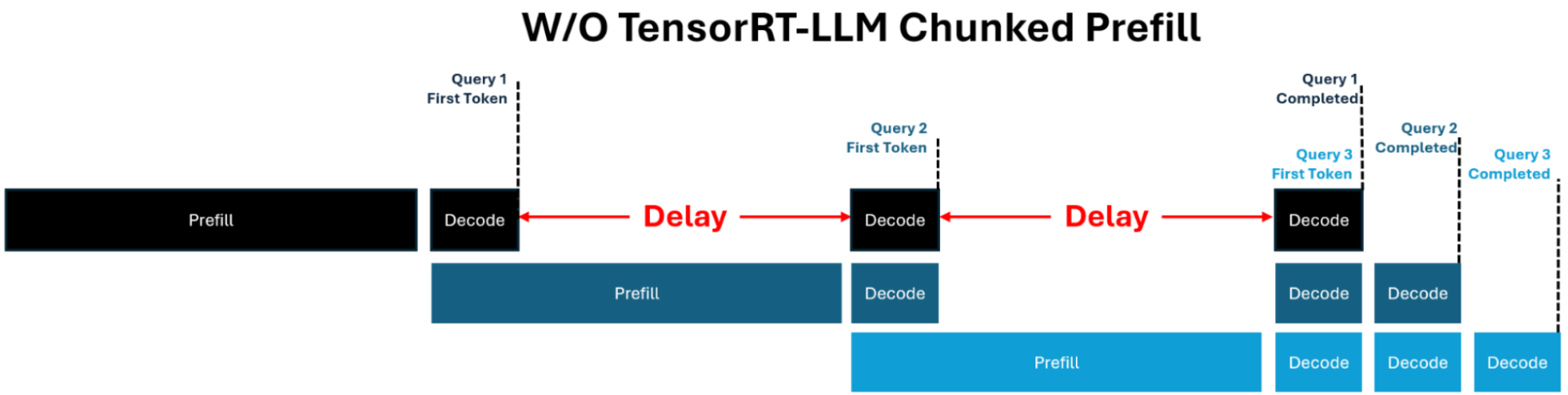
Chunked-Prefill in One Slide

Core idea: partition prefilling input into multiple chunks (e.g., 256 tokens/chunk) and mix with decoding requests

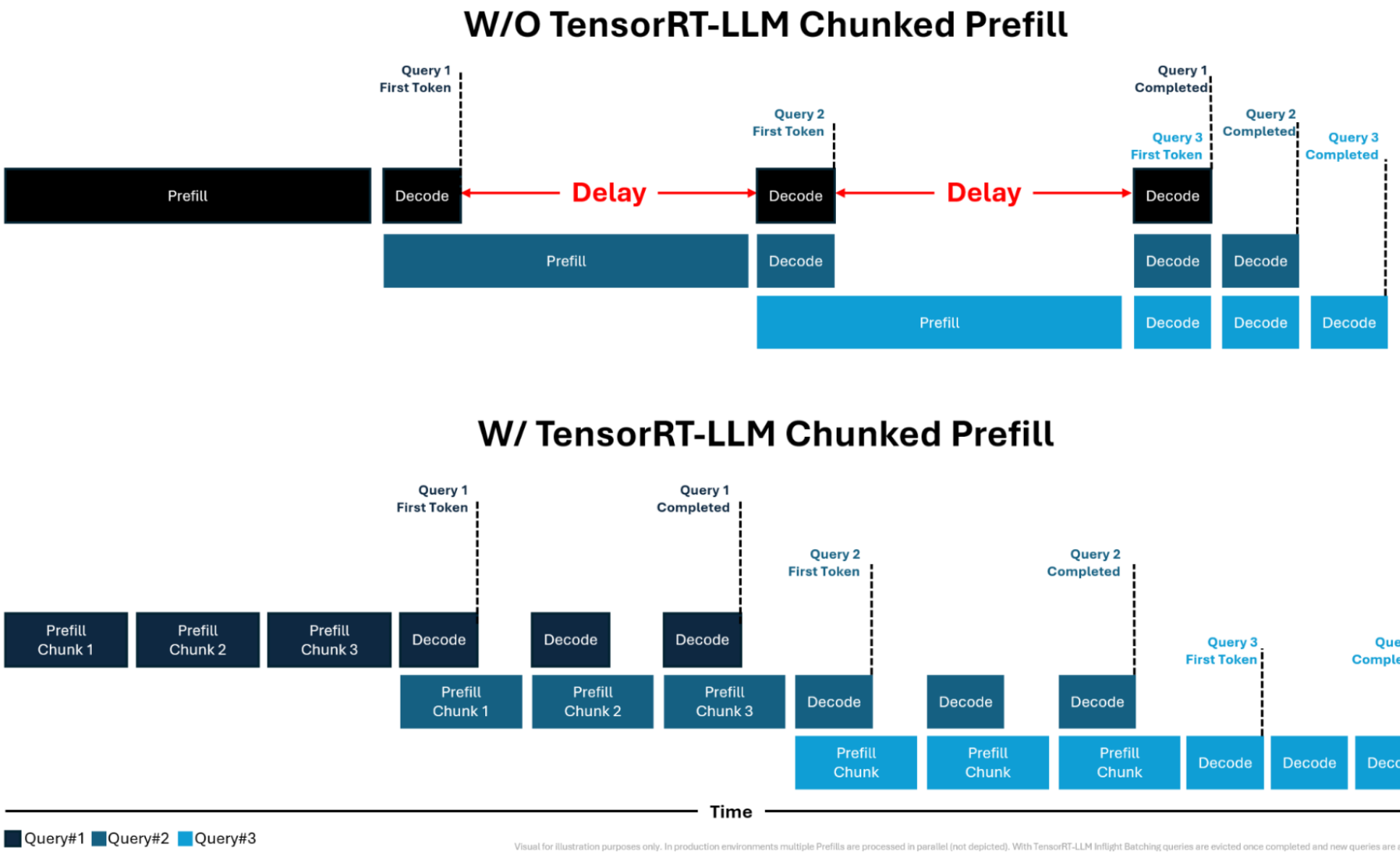


How about using a larger batch size?

Chunked-Prefill in One Slide



Chunked-Prefill in One Slide



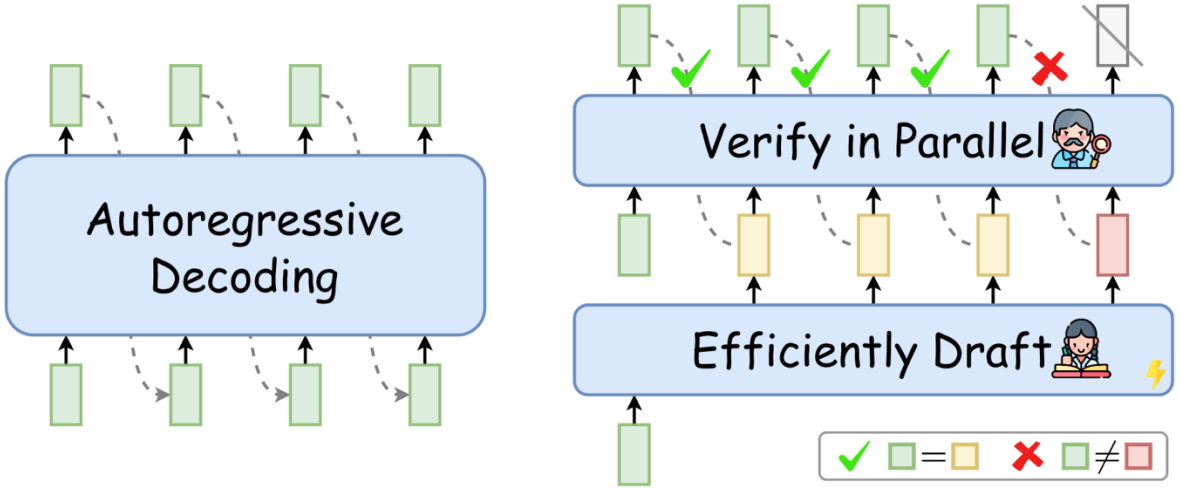
Visual for illustration purposes only. In production environments multiple Prefills are processed in parallel (not depicted). With TensorRT-LLM Inflight Batching queries are evicted once completed and new queries are added (not depictec).

Today

LLM Inference

- Basics (Prefilling and Decoding)
- Continuous Batching
- Chunked-Prefill

Panel Discussion: Speculative Decoding



Fast Inference from Transformers via Speculative Decoding, 2023

Questions?